JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent

Interface TransferQueue<E>

Type Parameters:

`E - the type of elements held in this collection`

All Superinterfaces:

`BlockingQueue<E>, Collection<E>, Iterable<E>, Queue<E>`

All Known Implementing Classes:

`LinkedTransferQueue`

```
public interface TransferQueue<E>
extends BlockingQueue<E>
```

A `BlockingQueue` in which producers may wait for consumers to receive elements. A `TransferQueue` may be useful for example in message passing applications in which producers sometimes (using method `transfer(E)`) await receipt of elements by consumers invoking `take` or `poll`, while at other times enqueue elements (via method `put`) without waiting for receipt. Non-blocking and time-out versions of `tryTransfer` are also available. A `TransferQueue` may also be queried, via `hasWaitingConsumer()`, whether there are any threads waiting for items, which is a converse analogy to a `peek` operation.

Like other blocking queues, a `TransferQueue` may be capacity bounded. If so, an attempted transfer operation may initially block waiting for available space, and/or subsequently block waiting for reception by a consumer. Note that in a queue with zero capacity, such as `SynchronousQueue`, `put` and `transfer` are effectively synonymous.

This interface is a member of the Java Collections Framework.

Since:

`1.7`

Method Summary

Abstract Methods

Modifier and Type	Method	Description
int	**getWaitingConsumerCount**()	Returns an estimate of the number of consumers waiting to receive elements via **BlockingQueue.take()** or timed **poll**.
boolean	**hasWaitingConsumer**()	Returns `true` if there is at least one consumer waiting to receive an element via **BlockingQueue.take()** or timed **poll**.
void	**transfer**(**E** e)	Transfers the element to a consumer, waiting if necessary to do so.
boolean	**tryTransfer**(**E** e)	Transfers the element to a waiting consumer immediately, if possible.
boolean	**tryTransfer**(**E** e, long timeout, **TimeUnit** unit)	Transfers the element to a consumer if it is possible to do so before the timeout elapses.

Methods inherited from interface java.util.concurrent.BlockingQueue

add, contains, drainTo, drainTo, offer, offer, poll, put, remainingCapacity, remove, take

Methods inherited from interface java.util.Queue

element, peek, poll, remove

Methods inherited from interface java.util.Collection

addAll, clear, containsAll, equals, hashCode, isEmpty, iterator, parallelStream, removeAll, removeIf, retainAll, size, spliterator, stream, toArray, toArray

Methods inherited from interface java.lang.Iterable

forEach

Method Detail

tryTransfer

```
boolean tryTransfer(E e)
```

Transfers the element to a waiting consumer immediately, if possible.

More precisely, transfers the specified element immediately if there exists a consumer already waiting to receive it (in `BlockingQueue.take()` or timed `poll`), otherwise returning `false` without enqueuing the element.

Parameters:

`e - the element to transfer`

Returns:

`true if the element was transferred, else false`

Throws:

`ClassCastException - if the class of the specified element prevents it from being added to this queue`

`NullPointerException - if the specified element is null`

`IllegalArgumentException - if some property of the specified element prevents it from being added to this queue`

transfer

```
void transfer(E e)
       throws InterruptedException
```

Transfers the element to a consumer, waiting if necessary to do so.

More precisely, transfers the specified element immediately if there exists a consumer already waiting to receive it (in `BlockingQueue.take()` or timed `poll`), else waits until the element is received by a consumer.

Parameters:

e - the element to transfer

Throws:

InterruptedException - if interrupted while waiting, in which case the element is not left enqueued

ClassCastException - if the class of the specified element prevents it from being added to this queue

NullPointerException - if the specified element is null

IllegalArgumentException - if some property of the specified element prevents it from being added to this queue

tryTransfer

```
boolean tryTransfer(E e,
                    long timeout,
                    TimeUnit unit)
             throws InterruptedException
```

Transfers the element to a consumer if it is possible to do so before the timeout elapses.

More precisely, transfers the specified element immediately if there exists a consumer already waiting to receive it (in `BlockingQueue.take()` or timed `poll`), else waits until the element is received by a consumer, returning `false` if the specified wait time elapses before the element can be transferred.

Parameters:

e - the element to transfer

timeout - how long to wait before giving up, in units of unit

unit - a TimeUnit determining how to interpret the timeout parameter

Returns:

true if successful, or false if the specified waiting time elapses before completion, in which case the element is not left enqueued

Throws:

InterruptedException - if interrupted while waiting, in which case the element is not left enqueued

ClassCastException - if the class of the specified element prevents it from being added to this queue

NullPointerException - if the specified element is null

IllegalArgumentException - if some property of the specified element prevents it from being added to this queue

hasWaitingConsumer

```
boolean hasWaitingConsumer()
```

Returns true if there is at least one consumer waiting to receive an element via BlockingQueue.take() or timed poll. The return value represents a momentary state of affairs.

Returns:

true if there is at least one waiting consumer

getWaitingConsumerCount

```
int getWaitingConsumerCount()
```

Returns an estimate of the number of consumers waiting to receive elements via BlockingQueue.take() or timed poll. The return value is an approximation of a momentary state of affairs, that may be inaccurate if consumers have completed or given up waiting. The value may be useful for monitoring and heuristics, but not for synchronization control. Implementations of this method are likely to be noticeably slower than those for hasWaitingConsumer().

Returns:

the number of consumers waiting to receive elements

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.